MODULE 15 · DOMAIN 2 - CONTAINER ORCHESTRATION

Getting traffic in: Ingress and the Gateway API

One entrance for many Services, the Ingress NGINX retirement, and what replaces it.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 15 of 35

admin@sthithapragna.com

One entrance, many Services: route by hostname and path, at the HTTP layer.

LoadBalancer Service versus an HTTP entrance

LOADBALANCER SERVICE

Layer 4

- One external address per Service
- Any TCP or UDP traffic
- Knows ports, not URLs
- Cost grows with every Service

INGRESS OR GATEWAY

Layer 7

- One address for many Services
- Routes by hostname and path
- Terminates TLS in one place
- One load balancer, shared

THE TIEBREAKER

Many HTTP services, one address, routing by host and path: Ingress or Gateway, not a LoadBalancer each.

Ingress: an API object and a controller

- **The Ingress object** lists rules: which host and path go to which Service.
- **An Ingress controller** makes it real. Kubernetes ships none. Without one, nothing happens.
- **Controllers include** Traefik, HAProxy, Contour, cloud load balancer controllers, and F5 NGINX.
- **The Ingress API is stable** since 1.19 and still supported, but feature-frozen.

> ON THE EXAM
>
> Ingress created, no address, no traffic: there is no Ingress controller running.

An Ingress for Quayside

```
 1 apiVersion: networking.k8s.io/v1
 2 kind: Ingress
 3 metadata: {name: quayside}
 4 spec:
 5   ingressClassName: traefik
 6   rules:
 7   - host: book.quayside.example
 8     http:
 9       paths:
10       - path: /api
11         pathType: Prefix
12         backend: {service: {name: booking-api, port: {number: 80}}}
13       - {path: /, pathType: Prefix, backend: {service: {name: web, port: {number: 80}}}}
```

5 Which controller should handle this Ingress.

10-13 /api goes to the booking API; everything else goes to web.

The parts of an Ingress

FIELD	PURPOSE
`ingressClassName`	Picks the controller, through an IngressClass object
`rules: host`	Match on the requested hostname
`paths: pathType`	Exact, Prefix, or ImplementationSpecific
`backend`	A Service name and port
`tls`	Hostnames plus a kubernetes.io/tls Secret holding the certificate
`defaultBackend`	Where unmatched requests go

ON THE EXAM

Ingress TLS comes from a kubernetes.io/tls Secret. The cert-manager tool often creates it.

Where Ingress falls short

- **HTTP and HTTPS only.** No TCP, UDP or gRPC-specific routing in the spec.
- **Anything beyond host and path** needs controller-specific annotations: rewrites, headers, canaries.
- **Annotations are not portable.** Switching controllers means rewriting them.
- **One object mixes** platform concerns, like TLS, with each app's routing.

> ON THE EXAM
>
> 'Portable canary weights without vendor annotations' is exactly what Ingress cannot offer.

Currency check: Ingress NGINX is retired

- **November 2025:** SIG Network and the Security Response Committee announced its retirement.
- **Since March 2026:** no further releases, no bug fixes, no security patches.
- **Existing installs still run,** but become less safe with every unpatched vulnerability.
- **The Ingress API is not deprecated.** Only this one community controller was retired.
- **F5's NGINX Ingress Controller** is a different, still-maintained product.

> ON THE EXAM
>
> Recommended path: move to a Gateway API implementation, or at least another Ingress controller.

The Gateway API is **role-oriented, expressive and portable**: the future of traffic into Kubernetes.

Gateway API resources

RESOURCE	OWNED BY	DEFINES
GatewayClass	Infrastructure provider	Which implementation: Envoy Gateway, Istio, a cloud…
Gateway	Cluster operator	Listeners: ports, protocols, hostnames, TLS
HTTPRoute	Application developer	Hosts, paths, headers and weights, to Services
GRPCRoute	Application developer	gRPC services and methods, to Services
ReferenceGrant	The target namespace	Permission for a cross-namespace reference

ON THE EXAM

Installed as CRDs, usually by the implementation. That is why its apiVersion is gateway.networking.k8s.io.

Who owns what

01

Infrastructure provider

Offers a GatewayClass.

02

Cluster operator

Creates a Gateway in the infra namespace, with TLS.

03

App teams

Write HTTPRoutes in their own namespaces.

04

Attach

Routes attach to the Gateway through parentRefs.

05

Services

Routes send traffic to ordinary Services.

ON THE EXAM

The platform team controls the entrance; each app team controls only its own routes.

An HTTPRoute with a canary, no annotations

```
 1 apiVersion: gateway.networking.k8s.io/v1
 2 kind: HTTPRoute
 3 metadata: {name: booking, namespace: quayside}
 4 spec:
 5   parentRefs: [{name: public, namespace: infra}]
 6   hostnames: [book.quayside.example]
 7   rules:
 8   - matches: [{path: {type: PathPrefix, value: /api}}]
 9     backendRefs:
10     - {name: booking-api, port: 80, weight: 90}
11     - {name: booking-api-canary, port: 80, weight: 10}
```

5 Attaches to the Gateway in infra, whose listener must allow this namespace.

10-11 90% to the stable version, 10% to the canary. Part of the spec.

Ingress versus Gateway API

INGRESS

Simple and frozen

- One object for everything
- HTTP and HTTPS
- Extras through vendor annotations
- Built in; stable; no new features

GATEWAY API

Expressive and growing

- Split by role across resources
- HTTP and gRPC; more protocols in progress
- Headers, weights, mirrors in the spec
- Installed as CRDs; actively developed

THE TIEBREAKER

Role separation, portable traffic splitting, or cross-namespace sharing: Gateway API.

Moving from Ingress to Gateway API

- **ingress2gateway,** a Kubernetes SIG tool, converts Ingress objects into Gateway API resources.
- **Many implementations support both,** so the two can run side by side during migration.
- **Popular implementations:** Envoy Gateway, Istio, Cilium, Traefik, Kong, NGINX Gateway Fabric.
- **Gateway API also reaches inside the cluster:** service meshes use it through GAMMA, in module 16.

> ON THE EXAM
>
> New work in Kubernetes networking happens in the Gateway API.

> On a fresh kubeadm cluster, an engineer creates an Ingress routing book.quayside.example to the web Service.
>
> kubectl shows the Ingress with no address, and requests never arrive. The web Service and its Pods are healthy.

What is the most likely cause?

A The backend Service must be type LoadBalancer

B The DNS record for the host has not been created

C No Ingress controller is running in the cluster

D The pathType must be Exact for a root path

◆ PAUSE AND DECIDE

What is the most likely cause?

A The backend Service must be type LoadBalancer

Backends are normally plain ClusterIP Services.

B The DNS record for the host has not been created

DNS would matter later, but the Ingress still gets no address.

C No Ingress controller is running in the cluster

Kubernetes ships none. No controller, no address, no traffic.

D The pathType must be Exact for a root path

Prefix with / is the usual catch-all.

THE SIGNAL

Fresh cluster, Ingress with no address: install an Ingress controller.

> Quayside currently runs two LoadBalancer Services, one for web and one for the booking API, and pays for two cloud load balancers.
>
> They want one public address: book.quayside.example/api to the API, everything else to web.

What should they do?

A One Ingress or Gateway, routing by path to both Services

B Change both Services to NodePort and publish node addresses

C Make both Services ExternalName pointing at the same host

D Make both Services headless and publish the Pod addresses

◆ PAUSE AND DECIDE

What should they do?

A One Ingress or Gateway, routing by path to both Services

One entrance, path routing, one load balancer to pay for.

B Change both Services to NodePort and publish node addresses

No path routing, and users would chase node IPs.

C Make both Services ExternalName pointing at the same host

ExternalName points outwards; it serves nothing.

D Make both Services headless and publish the Pod addresses

Headless is internal, and Pod IPs change constantly.

THE SIGNAL

One address, routing by path to several Services: an HTTP entrance.

> In September 2026, Quayside's security team finds the community ingress-nginx controller still running in two clusters.
>
> They ask for an accurate statement of its status and the right response.

Which statement is accurate?

A The Ingress API is deprecated and will soon be removed from Kubernetes

B It is retired and unpatched since March 2026; plan a migration

C It still gets security patches until Kubernetes 2.0 is released

D Every Ingress it serves has already stopped working

◆ PAUSE AND DECIDE

Which statement is accurate?

A The Ingress API is deprecated and will soon be removed from Kubernetes

The API is stable and frozen, not deprecated. Only the controller retired.

B It is retired and unpatched since March 2026; plan a migration

Still runs, gets no fixes. Move to Gateway API or another controller.

C It still gets security patches until Kubernetes 2.0 is released

No patches of any kind since March 2026.

D Every Ingress it serves has already stopped working

Existing installations keep running; they are just unmaintained.

THE SIGNAL

Controller retired, API alive. Running but unpatched means migrate, not panic.

> Quayside's platform team wants to own the public entrance and its TLS certificates. Each app team should manage only its own routing, in its own namespace.
>
> The booking team also wants to send 10% of traffic to a canary, without vendor-specific annotations.

Which approach fits?

A One Ingress per team, with controller annotations for the canary

B A LoadBalancer Service for each team's applications

C NodePort Services, with the platform team's own proxy in front

D A shared Gateway, with each team's HTTPRoutes using weights

◆ PAUSE AND DECIDE

Which approach fits?

A One Ingress per team, with controller annotations for the canary

Uses the very vendor annotations they want to avoid.

B A LoadBalancer Service for each team's applications

Many entrances, no shared TLS, no weighted routing.

C NodePort Services, with the platform team's own proxy in front

Hand-built plumbing that the Gateway API already provides.

D A shared Gateway, with each team's HTTPRoutes using weights

Role split by design; weights in the standard spec.

THE SIGNAL

Separate roles plus portable weights: Gateway and HTTPRoute.

Module 15 in five lines

- **One HTTP entrance for many Services,** routing by host and path, instead of a LoadBalancer each.
- **Ingress needs a controller;** Kubernetes ships none. The API is stable and frozen.
- **Ingress NGINX is retired:** no patches since March 2026. The Ingress API is not deprecated.
- **Gateway API:** GatewayClass, Gateway, HTTPRoute, split by role, installed as CRDs.
- **Weights and headers in the spec,** portable across implementations. ingress2gateway converts.

That is traffic from outside. Next: **service meshes**, for traffic between services inside the cluster.